Tareas del proyecto

Lista de lo que hay que construir, derivada del enunciado y la rúbrica (`docs/document.pdf`).
Marcar con `[x]` al completar y anotar quién lo toma. Cada bloque se puede trabajar en paralelo.

Entrega: 8 de noviembre de 2026.

0. Preparación (todos)

- [x] Clonar el repo, copiar `.env.example` a `.env` y comprobar que `docker compose up` abre http://localhost:8080
- [x] Acordar reparto de los bloques 1 a 4

1. Autenticación (rúbrica: sistema de autenticación seguro)

- [x] Modelo `Usuario` (`app/Models/Usuario.php`)
- [x] `AuthController`: mostrar login, procesar login, logout
- [x] Vistas `app/Views/auth/login.php` y layout `auth.php`
- [x] Contraseñas con `password_hash` / `password_verify`
- [x] Helper `Auth` que redirige a `/login` si no hay sesión (todas las rutas protegidas por defecto)
- [x] Protección CSRF en formularios (`Helpers/Csrf.php`)
- [x] Pruebas: `tests/UsuarioTest.php`, `AuthTest.php`, `CsrfTest.php`, `HttpAuthTest.php`
- [ ] Opcional: registro de usuarios desde el panel admin y roles (admin / recepción)

2. Módulo Huéspedes - CRUD (requisito central del enunciado)

- [ ] Modelo `Huesped` con `create`, `update`, `all`, `find`, `delete`
- [ ] `HuespedController`: `index`, `create`, `store`, `edit`, `update`, `destroy`
- [ ] Vistas: `index.php` (tabla), `crear.php`, `editar.php` (pueden compartir un `_form.php`)
- [ ] Validación servidor: campos obligatorios, documento único, email válido, edad numérica
- [ ] Validación cliente en JS (`public/assets/js/huespedes.js`): mismos campos, sin recargar página
- [ ] Búsqueda de huésped por documento vía AJAX (reutilizable al crear reservas)

PROF

Campos del huésped: nombre completo, tipo de documento, número de documento, dirección, teléfono, celular, edad, correo electrónico, contacto de emergencia, parentesco del contacto.

3. Módulo Reservas (registrar, consultar, modificar, cancelar)

- [ ] Modelos `Reserva`, `Habitacion`, `TipoHabitacion`
- [ ] `ReservaController`: `index`, `create`, `store`, `edit`, `update`, `cancel`
- [ ] Vistas: listado con filtros (por fecha, por huésped, por estado), formulario crear/editar
- [ ] Lógica de disponibilidad: no permitir dos reservas activas de la misma habitación con fechas que se crucen
- [ ] Endpoint AJAX `/api/habitaciones/disponibles?checkin=&checkout=&tipo=` que devuelva JSON

- [ ] Validación de fechas: checkout posterior a checkin, no reservar en el pasado
- [ ] Historial de reservas por huésped

Datos de la reserva: tipo de habitación, fecha de ingreso, fecha de salida, número de huéspedes.

4. Diseño visual (rúbrica 20%: HTML5 y CSS3)

- [ ] Layout general responsive (`app/Views/layouts/main.php` + `styles.css`)
- [ ] Estilo de tablas, formularios, botones y mensajes de error/éxito
- [ ] Página de login
- [ ] Dashboard de inicio con resumen (reservas de hoy, habitaciones libres)

5. Base de datos (rúbrica 20%: relacional y no relacional)

- [ ] Revisar `database/schema.sql` con el grupo y ajustar si falta algo
- [ ] Diagrama entidad-relación en `docs/` (imagen o Mermaid)
- [ ] Parte "no relacional": decidir cómo cubrirla. Opción sencilla: guardar un log de acciones (quién creó/modificó/canceló cada reserva) en un documento JSON o en MongoDB. Documentar la decisión.

6. Documentación y entrega (rúbrica 20% entre documento y video)

- [ ] Documento Word/PDF: portada con integrantes y código del curso, descripción del trabajo, arquitectura, capturas, link al Drive con el código y link al video
- [ ] Video de máximo 5 minutos mostrando: login, CRUD de huéspedes, crear/modificar/cancelar reserva, validaciones
- [ ] Comprimir el código fuente (sin `docker/mysql/data` ni `.env`) y subirlo a Drive
- [ ] Guardar el documento y el enlace del video en `docs/entrega/`

Reparto de tareas

Persona	Bloques
A	1 Autenticación + 4 Diseño visual
B	2 Huéspedes + validaciones JS/AJAX
C	3 Reservas + 5 Base de datos

Bloque 6 lo hacemos todos al final. Cada persona documenta su parte a medida que avanza.

PROF